Name: _____ Student NUID: _____

CS 2100: Program Design and Implementation 1

Practice Quiz 3

Instructions

- Do not begin the quiz until instructed to do so.
- You may use both sides of a sheet of paper up to 8.5"x11" for reference, but no other resources, including phones, computers, AI, headphones, and ear pods.
- You have until the end of the class period to complete the quiz.
- Students may not leave the classroom during the first 10 minutes of the quiz (except in case of emergency).
- Hand your completed quiz to an instructor before leaving the room.
- Talk to an instructor if you need to leave the room and reenter.

1. Unit testing

The following code has a bug in it. Please fill in a unit test that will fail on this buggy code, but will pass if the code is updated to correctly fit the documentation. You only need to write the one test.

```
def lowest(nums: list[float]) -> float:
    """Returns the lowest number from the list.

    Args:
        nums (list[float]): the list of numbers from which
                we want the lowest

    Returns:
        float: the lowest number in the list

    Raise:
        ValueError if nums is empty
    """
    if not nums:
        raise ValueError(
            f'Cannot find lowest number in empty list')
    return nums[0] # HINT: the bug is in this line

class TestLowest(unittest.TestCase):
    """Tests for lowest()"""
    def test_this_bug(self) -> None:
        """Student's test to catch the bug"""
```

Solution:

```
        self.assertAlmostEqual(3.0, lowest([4.0, 3.0]))
```

2. Classes

Consider the code below. Please complete the `add_student()` method to match the documentation.

```
class Student:
    def __init__(self, student_id: str, name: str):
        """Instantiates a Student with the given student ID and name.

        Args:
            student_id (str): the student's assigned ID
            name (str): the student's name
        """
        self.__student_id = student_id
        self.name = name

    @property
    def student_id(self) -> str:
        """Returns the student ID"""
        return self.__student_id

    def __str__(self) -> str:
        """Returns the student's name"""
        return self.name

class Club:
    """A student club"""
    def __init__(self, name: str):
        """Instantiates a new club

        Args:
            name (str): the name of the club
        """
        self.name = name
        self.__students: list[Student] = []

    @property
    def students(self) -> list[Student]:
        """Returns the list of students in the club"""
        return self.__students.copy()

    def add_student(self, student: Student) -> None:
        """Adds the given student to the club

        Args:
            student (Student): the student to add to the club
        """
```

Solution:

```
        self.__students.append(student)
```

3. Using objects

Consider the code from Question 2, partially copied below:

```
class Student:
    def __init__(self, student_id: str, name: str):
        """Instantiates a Student with the given student ID and name.

        Args:
            student_id (str): the student's assigned ID
            name (str): the student's name
        """
        self.__student_id = student_id
        self.name = name

    @property
    def student_id(self) -> str:
        """Returns the student ID"""
        return self.__student_id

    def __str__(self) -> str:
        """Returns the student's name"""
        return self.name
```

What is the output of the following code?

```
blep = Student('S1234', 'Blep')
blep.name = 'Mlem'
print(blep)
```

Solution:

```
Mlem
```

4. Lists, sets, and dictionaries

Considering the code from Question 2 (partially copied below), please complete the `get_student_id_set()` method to match the documentation. Notice that it returns a set.

```
class Student:
    def __init__(self, student_id: str, name: str):
        """Instantiates a Student with the given student ID and name.

        Args:
            student_id (str): the student's assigned ID
            name (str): the student's name
        """
        self.__student_id = student_id
        self.name = name

    @property
    def student_id(self) -> str:
        """Returns the student ID"""
        return self.__student_id
    ...

class Club:
    def __init__(self, name: str):
        """Instantiates a new club

        Args:
            name (str): the name of the club
        """
        self.name = name
        self.__students: list[Student] = []

    @property
    def students(self) -> list[Student]:
        """Returns the list of students in the club"""
        return self.__students.copy()

    def get_student_id_set(self) -> set[str]:
        """Returns a set of the student IDs of all students in
        the club
        """
```

Solution:

```
        return {student.student_id for student in self.__students}
```

5. Properties

Consider the following code. In the `Course` class please add a setter for the `department` property. The setter should update the department if it is a valid department in the set of `DEPARTMENTS` above. If it is not a valid department, it should raise a `ValueError`.

```
DEPARTMENTS = {
    'Khoury', 'CAMD', 'Bouve', 'Engineering', 'Mills',
    'Sciences', 'CPS', 'CSSH', 'DMSB', 'Law'
}

class Course:
    """A course available in the university registration system."""

    def __init__(
        self, name: str, department: str, start_time: str):
```

```
        """
        Initializes a course in the registration system.

        Args:
            name (str): the name of the course
            department (str): the department of the course
            start_time (str): the time when the course meets

        Raises:
            ValueError if department is not a valid department
                    at the university
        """
        self.dept = department
        self.name = name
        self.start_time = start_time
        self.__students: set[Student] = set()

    def check_dept(self, department: str) -> None:
        if department not in DEPARTMENTS:
            raise ValueError(f'Invalid department: {department}')

    def __eq__(self, other: object) -> bool:
        """Return True if other is a Course with the same
        name and department, and False otherwise"""
        if not isinstance(other, Course):
            return False
        return self.name == other.name and \
            self.dept == other.dept

    def __str__(self) -> str:
        return f'{self.dept}: {self.name}'

    @property
    def dept(self) -> str:
        """Return the department of the course"""
        return self._department
```

Solution:

```
    @dept.setter
    def dept(self, new_dept: str) -> None:
        check_dept(new_dept)
        self._department = new_dept
```

```
    @property
    def students(self) -> set[Student]:
        """Returns the list of students registered for the course"""
        return self.__students.copy()

    def register_student(self, student: Student) -> None:
        """Adds the student to the course.

        Args:
            student (Student): the student to add to the course
        """
        self.__students.add(student)
```

6. Inheritance

Consider the following subclasses of `Course` (from Q5). Please fill in the `__eq__()` method to make it so that a `GraduateCourse` can only equal a `GraduateCourse`, if they have the same name and department.

```
class UndergraduateCourse(Course):
    """A course specific to undergraduate students."""

class GraduateCourse(Course):
    """A course specific to graduate students."""

    def __eq__(self, other: object) -> bool:
        """Returns True if other is a GraduateCourse
        with the same name and department, and False
        otherwise.

        Args:
            other (object): the object for comparison

        Returns:
            True if other is a GraduateCourse with the
            same name and department, and False otherwise
        """
```

Solution:

```
        if isinstance(other, GraduateCourse):
            return super().__eq__(other)
        return False
```

OR

```
        if isinstance(other, GraduateCourse):
            return self.name == other.name and \
                self.dept == other.dept
        return False
```

7. Coupling, cohesion, and encapsulation

Recall: the Single Responsibility Principle states that each unit (class, function, variable, etc.) should have a single, well-defined responsibility.

Please select all statements below that are true. You may circle the letter for the sentence, or circle the entire sentence. If you accidentally circle something you don't want to, and you cannot erase it, please cross it out by putting a line through it.

A. Adding a `print_course()` method to the `Course` class will make `Course`'s encapsulation WORSE.

B. Making `self.students` an attribute instead of a property (and getting rid of `self.__students`) will make `Course`'s encapsulation WORSE.

C. Making `Course`'s `students` property return a direct reference to the list of students (instead of a copy of the list) will make `Course`'s encapsulation WORSE.

D. High cohesion requires following the Single Responsibility Principle.

E. We should always prioritize cohesion, even at the expense of encapsulation.

Solution:

B, C, D